

Lecture 10 一推理 (Inference) 学习讲义

CS336 · 基于 inference.py 的逐节精读

整理自 Stanford CS336 Lecture 10

2026-07-01

Contents

0. 一句话主线	3
1. 推理的全景与指标	4
1.1 为什么效率至关重要	4
1.2 三个核心指标 (务必区分)	4
1.3 为什么推理天生比训练难优化	4
2. 维度记号约定 (einops 风格)	5
3. 地基: 算术强度 (Arithmetic Intensity)	6
3.1 定义	6
3.2 用一个矩阵乘法把它算清楚	6
3.3 硬件判据: 和 H100 的比值对比	6
3.4 极端情况 $B = 1$ (矩阵 $\times$ 向量)	6
4. 推理的两个阶段	7
4.1 KV Cache 的动机	7
4.2 两个阶段命运完全不同	7
5. 对 MLP 与 Attention 分别算强度	8
5.1 MLP 层 (只看矩阵乘法)	8
5.2 Attention 层 (FlashAttention, 只看矩阵乘法)	8
5.3 【最关键洞察】为什么 batch 救不了 attention	8
5.4 小结表	9
6. Latency vs Throughput 的根本权衡	10
6.1 关键公式 (来自代码)	10
6.2 三档 batch 的实验结论	10
6.3 权衡的本质	10
6.4 实践推论	10
7. 有损优化之一: 缩小 KV Cache	11
7.1 GQA (Grouped-Query Attention)	11
7.2 MLA (Multi-head Latent Attention, DeepSeek)	11
7.3 CLA (Cross-Layer Attention)	11
7.4 局部 / 滑动注意力 (Sliding Window)	11
7.5 DeepSeek v4 注意力 (支持 1M 上下文)	11
7.6 本节小结	11
8. 有损优化之二: 量化 (Quantization)	12
8.1 机制	12

8.2 精度格式	12
8.3 两种路线	12
8.4 AWQ (Activation-aware Weight Quantization)	12
9. 有损优化之三：剪枝 + 蒸馏	13
10. 无损优化：推测采样 (Speculative Sampling)	14
10.1 利用的核心不对称	14
10.2 算法	14
10.3 两元素证明 (词表 = {A, B})	14
10.4 实践与扩展	14
10.5 小结	14
11. 处理动态负载 (系统技巧)	15
11.1 Continuous Batching (Orca)	15
11.2 PagedAttention (vLLM)	15
12. 全局总结	16
12.1 一图流	16
12.2 要点清单	16
12.3 最容易考/最该记的三件事	16

0. 一句话主线

训练是 **compute-bound**，推理是 **memory-bound**。

这一节课所有的技巧，本质都在做两件事之一：（1）减少要搬运的字节数，或（2）把串行生成变成可并行的验证。

把这句话钉在脑子里，后面每个技巧都能对号入座。

整个 `inference.py` 的结构（对应课程主线）：

代码函数	讲什么	属于
<code>landscape()</code>	推理为什么重要、指标定义	背景
<code>review_transformer()</code>	维度记号约定	铺垫
<code>review_of_arithmetic_intensity()</code>	算术强度的定义与判据	核心地基
<code>arithmetic_intensity_of_inference()</code>	MLP / Attention 分别算强度	核心
<code>throughput_and_latency()</code>	latency vs throughput 权衡	核心
<code>reduce_kv_cache_size()</code>	GQA / MLA / CLA / 局部注意力	有损优化
<code>quantization()</code>	量化	有损优化
<code>model_pruning()</code>	剪枝 + 蒸馏	有损优化
<code>speculative_sampling()</code>	推测采样	无损优化
<code>continuous_batching()</code> / <code>paged_attention()</code>	动态负载工程	系统

1. 推理的全景与指标

1.1 为什么效率至关重要

训练是一次性成本，推理是重复无数次的成本。课程给的对比很有冲击力：

- OpenAI 每天处理约 **8.6T tokens**
- DeepSeek v4 整个训练只用了 **32T tokens**

也就是说 **OpenAI** 约 **4 天** 的推理吞吐量 $\approx$ 训练一个前沿模型的全部数据量。推理效率直接等于成本。

补充观察：

- **Chatbot**：多数 token 给人看，人是瓶颈（慢一点没关系）。
- **Agent**：query $\rightarrow$ 内部推理链 $\rightarrow$ 输出，生成 token 数可以无上限增长。
- 生成的 **token 数** = 花掉的算力。

1.2 三个核心指标（务必区分）

指标	含义	关心它的场景	主要由谁决定
TTFT (time-to-first-token)	用户等多久看到第一个字	交互式应用	prefill 时间
Latency (秒/token)	单条请求出字速度	交互式应用	KV cache 大小
Throughput (token/秒)	多条请求的总吞吐	批处理	batch 大小

1.3 为什么推理天生比训练难优化

- **训练（监督）**：一次能看到所有 token，可以在序列维度上并行 $\rightarrow$ 变成大矩阵乘法 (compute-bound, 好)。
- **推理**：必须逐个 **token 顺序生成**，无法在生成维度上并行 $\rightarrow$ 很难把算力吃满 (memory-bound, 差)。

2. 维度记号约定 (einops 风格)

后面所有公式都用这套字母，先记熟：

符号	含义
B	batch (并发请求数)
S	已有 / 条件 token 数 (历史长度)
T	要生成 / 预测的 token 数
D	模型隐藏维 (model dim)
F	MLP 上投影维, 通常 $F = 4D$
N	query head 数
K	key/value head 数 (GQA 用)
H	每个 head 的维度, $D = N \cdot H$
L	层数
V	词表大小

矩阵乘法记号: $\text{BTD} \times \text{DH} \rightarrow \text{BTH}$, 其中红色维度 (这里是 D) 在两个操作数里都出现、在结果里消失, 叫收缩维 (**contracting**); 黑色维度只在一边出现、保留到结果。

约定关系: $F = 4D$, $D = NH$, $N = KG$ (GQA 把 N 个 head 分成 K 组), 训练时 $S = T$ 。

3. 地基：算术强度 (Arithmetic Intensity)

这是理解整节课的钥匙。

3.1 定义

$$\text{intensity} = \frac{\text{FLOPs}}{\text{bytes transferred}}$$

即每搬运一个字节，能做多少次浮点计算。越高越好（说明算力被充分利用）。

3.2 用一个矩阵乘法把它算清楚

设 X 形状 $(B \times D)$, W 形状 $(D \times F)$, 计算 $Y = XW$ 。逐步做 FLOPs 与访存记账 (bf16, 每个数 2 字节):

步骤	操作	FLOPs	Bytes
1	从 HBM 读 $X (B \times D)$		$2BD$
2	从 HBM 读 $W (D \times F)$		$2DF$
3	计算 $Y = X@W$	$2BDF$	
4	把 $Y (B \times F)$ 写回 HBM		$2BF$

于是:

$$\text{intensity} = \frac{2BDF}{2BD + 2DF + 2BF}$$

当 B 远小于 D, F 时 (令 $D = cB$, $F = cB$, $c \rightarrow \infty$) 化简得:

$$\boxed{\text{intensity} \rightarrow B}$$

直觉: 算术强度约等于 batch size。

3.3 硬件判据: 和 H100 的比值对比

$$\text{accelerator intensity} = \frac{\text{FLOP/s}}{\text{memory bandwidth}} = \frac{989 \times 10^{12}}{3.35 \times 10^{12}} \approx 295$$

判据:

- 计算强度 $> 295 \rightarrow$ **compute-bound** (好, 卡在算力)
- 计算强度 $< 295 \rightarrow$ **memory-bound** (差, 卡在搬数据)
- 对上面的矩阵乘法: **compute-bound** 当且仅当 $B > 295$ 。

3.4 极端情况 $B = 1$ (矩阵 $\times$ 向量)

- 算术强度 $= 1$
- 严重 memory-bound: 读了一个 $D \times F$ 的大矩阵, 却只做了 $2DF$ 次计算
- 这几乎就是推理逐 **token** 生成时发生的事。

4. 推理的两个阶段

4.1 KV Cache 的动机

朴素推理：每生成一个 token，都把完整历史喂进 Transformer。

- 一次前向 $O(T^2)$ ，生成 T 个 token 需要 $O(T^3)$ FLOPs。

观察：大量计算在不同前缀之间是可共享的。方案：把 **KV cache** 存在 HBM 里。

KV cache：对每个序列 (B)、每个 token (S)、每层 (L)、每个 KV head (K)，存一个 H 维向量。

4.2 两个阶段命运完全不同

阶段	做什么	参数设置	特性
Prefill	把 prompt 一次性编码成向量 (可像训练一样并行)	$T = S$	compute-bound (好)
Generation	逐 token 生成响应 (串行)	$T = 1$	memory-bound (差)

5. 对 MLP 与 Attention 分别算强度

设 S = 条件 token 数, T = 要生成的 token 数。之后特化到 prefill ($T = S$) 与 generation ($T = 1$)。

5.1 MLP 层 (只看矩阵乘法)

包含三个权重 $W_{up}(D \times F)$ 、 $W_{gate}(D \times F)$ 、 $W_{down}(F \times D)$ 。记账结果:

$$\begin{aligned} \text{FLOPs} &= 6BTDF \\ \text{Bytes} &= 4BTD + 4BTF + 6DF \end{aligned}$$

当 $BT \ll D, F$ 时化简:

$\text{MLP intensity} \rightarrow B \cdot T$

两阶段:

- **Prefill**: 把 $B \cdot T$ (大 batch、长序列) 做大就能 compute-bound $\rightarrow$ 好。
- **Generation**: 两个问题
 - 一次只生成一个 token ($T = 1$)
 - B 是并发请求数, 交互式应用里不可控 $\rightarrow$ 强度只剩 B , 要靠攒并发救。

5.2 Attention 层 (FlashAttention, 只看矩阵乘法)

- S = 之前已生成的 token 数
- T = 接下来要生成 logits 的 token 数

记账结果:

$$\text{FLOPs} = 4BSTD, \quad \text{Bytes} = 4BSD + 4BTD$$

$\text{Attention intensity} = \frac{ST}{S+T}$

两阶段:

- **Prefill** ($T = S$): 强度 = $\frac{S}{2} \rightarrow$ 好。
- **Generation** ($T = 1$): 强度 = $\frac{S}{S+1} < 1 \rightarrow$ 坏, 无法改善。

5.3 【最关键洞察】为什么 batch 救不了 attention

注意 attention 强度里没有 B ! 所以攒 batch 对 generation 的 attention 毫无帮助。

原因:

- **MLP 层**: 每条序列都打到同一套 MLP 权重 ($W_{up}, W_{gate}, W_{down}$ 不含 B) $\rightarrow$ 攒 batch 能把“读权重”的成本摊薄。
- **Attention 层**: 每条序列有自己的 KV cache (Q, K, V 都带 B 维) $\rightarrow$ batch 越大要读的 KV 越多, 摊不开。

5.4 小结表

	Prefill	Generation
MLP 强度	$B \cdot S$ (易 compute-bound)	B (需并发)
Attention 强度	$S/2$ (好)	< 1 (无解)
总体	compute-bound	memory-bound

6. Latency vs Throughput 的根本权衡

代码用 `compute_transformer_performance_stats` 对 **Llama 2 13B / H100** 建模。

6.1 关键公式（来自代码）

参数量：

$$\text{num_params} = 2VD + 3DFL + (2DNH + 2DKH)L$$

- 参数占内存 = $2 \times \text{num_params}$ (bf16)
- 每条序列的 KV cache = $S \cdot (KH) \cdot L \cdot 2 \cdot 2$ (一个 2 是 key+value, 另一个 2 是 bf16)
- 总内存 = $B \times \text{KV cache/seq} + \text{参数内存}$

$$\text{Latency} = \frac{\text{memory}}{\text{bandwidth}}$$

$$\text{Throughput} = \frac{B}{\text{Latency}}$$

直觉：每生成一步，都要把**全部参数** + **全部 KV cache** 读一遍 → latency 由访存量决定。

6.2 三档 batch 的实验结论

Batch	Latency	Throughput	备注
1	最低	最低	单请求最快
64	变差	变好	
256	更差	更好但收益递减	KV cache 撑爆 80GB 显存!

6.3 权衡的本质

- **batch** ↑ → **latency** 变差: KV cache 是 $O(B)$, 越大读写越慢。
- **batch** ↑ → **throughput** 变好: 把”读参数”的固定成本摊到更多序列上。

小 batch ⇒ 低延迟、低吞吐 大 batch ⇒ 高吞吐、高延迟

6.4 实践推论

- 简单并行: 起 M 份模型副本 → latency 不变, throughput $\times M$ 。
- 困难并行: 切分模型和 KV cache (sharding)。
- **TTF**T 主要是 **prefill** 时间 → **prefill** 用小 **batch** (TTF**T** 快), **generation** 用大 **batch** (吞吐高)。

7. 有损优化之一：缩小 KV Cache

推理是 memory-bound, KV cache 又随 B 、随序列长度增长, 所以第一刀砍它——但要保证精度不掉太多。

7.1 GQA (Grouped-Query Attention)

- 思路: N 个 query head, 但只有 K 个 KV head, 每个 KV head 服务 N/K 个 query head。
- 特例: MHA 是 $K = N$; MQA 是 $K = 1$; GQA 介于两者之间。
- **KV cache** 缩小 N/K 倍 $\rightarrow$ 因为 memory-bound, 直接提速。
- 代码演示: K 从 40 降到 8 (1:5), KV cache 变小 $\rightarrow$ 能塞进显存, 还能把 batch 加大到 256。精度基本不掉。

注意: 单看同一 batch, GQA 会让 latency 略差 (怎么会? 因为对比是“降 K 后可以加大 batch”, 加大 batch 换吞吐, 代价是延迟)——但整体在 pareto 前沿上更优。

7.2 MLA (Multi-head Latent Attention, DeepSeek)

- 普通注意力: KV cache 存 $K = W_K h$ 、 $V = W_V h$, 共 $N \cdot H$ 维。
- **MLA**: 只存一个压缩向量 $c = W_c h$ (C 维), 用时再投影回 $K = W_K c$ 、 $V = W_V c$ 。
- DeepSeek v2: 把 $N \cdot H = 16384$ 压到 $C = 512$ 。
- 坑: MLA 与 RoPE 不兼容, 需额外加 64 维放 RoPE $\rightarrow 512 + 64 = 576$ 维。
- 精度: MHA $>$ GQA (更贵但更准); **MLA 甚至略优于 MHA, 且便宜得多。**

7.3 CLA (Cross-Layer Attention)

- 思路: 跨层共享 KV (正如 GQA 跨 head 共享)。
- 经验上改善了“精度 vs KV cache 大小”的 pareto 前沿。

7.4 局部 / 滑窗注意力 (Sliding Window)

- 思路: 只看局部上下文 (对建模最相关的部分)。
- 有效上下文随层数线性增长; **KV cache** 不再依赖序列长度。
- 问题: 仍会掉精度。方案: 局部注意力与全局注意力交错 (hybrid 层)。

7.5 DeepSeek v4 注意力 (支持 1M 上下文)

- **CSA** (Compressed Sparse Attention): 每 m 个 token 压成 1 个。
- **DSA** (DeepSeek Sparse Attention): 选 top- k 。
- **HCA** (Heavily Compressed Attention): 压得更狠。

7.6 本节小结

- 目标: 缩小 KV cache (因为 memory-bound) 而不伤精度。
- 降维: GQA / MLA / CLA。
- 截断: 局部注意力 (部分层)。
- 其它方向: 线性注意力 / 状态空间模型 (Mamba 2、GatedDeltaNet)、扩散模型。

8. 有损优化之二：量化 (Quantization)

核心：降低数字精度 → 更少内存 → 更高吞吐/更低延迟（因为 memory-bound）。当然要担心精度。

8.1 机制

$$x_{quant} = \text{round}(x/\text{scale}) + \text{zero_point}, \quad x_{approx} = (x_{quant} - \text{zero_point}) \cdot \text{scale}$$

8.2 精度格式

格式	字节	用途
fp32	4	训练时参数和优化器状态
bf16	2	推理默认
fp8 (e4m3, H100)	1	敢的话能训练
int8	1	比 fp8 便宜但更不准，仅推理
int4	0.5	更便宜、更不准

8.3 两种路线

- **QAT (量化感知训练)**：训练时前向做“量化-反量化”模拟误差。
 - 优点：权重被训练得适应量化；缺点：需昂贵的大规模训练。
- **PTQ (训练后量化)**：训练完再做，便宜。用样本数据确定每层/张量的 scale 和 zero point。
 - **GPTQ**：用 Hessian 信息调整未量化权重，补偿量化误差。

8.4 AWQ (Activation-aware Weight Quantization)

- 观察：某些激活通道数值很大 → 打到这些通道的权重更重要。
- 思路：根据激活值，挑出 **0.1%–1%** 的重要权重保留高精度。
- 效果：fp16 → int3，内存降 4×，提速 3.2×。

9. 有损优化之三：剪枝 + 蒸馏

核心：直接把昂贵模型的一部分”拆掉”让它变便宜，再修复。

NVIDIA 的算法：

1. 在小校准集（1024 样本）上识别重要的 {layer, head, hidden dim}。
2. 移除不重要的层，得到更小的模型。
3. 把原模型蒸馏进剪枝后的模型（修复精度）。

两种整体配方：

- 从头训练：定义更快的架构 → 训练。
- 蒸馏：定义更快架构 → 用原模型（架构不同）初始化权重 → 蒸馏修复。

10. 无损优化：推测采样 (Speculative Sampling)

前面都是有损的。推测采样厉害在完全无损。

10.1 利用的核心不对称

检查比生成快。

Prefill 能并行地给出每个位置的概率 (compute-bound)，而 generation 只能逐个吐 (memory-bound)。

10.2 算法

- 用一个便宜的 **draft model** p 猜几个 token (比如 4 个)。
- 用 **target model** q 并行验证这些 token，用改良的拒绝采样接受/拒绝。
- 改良点：至少生成一个候选 (否则拒绝采样会一直循环)。
- 关键性质：输出保证是 **target model** 的精确样本 (exact sample)!

10.3 两元素证明 (词表 = {A, B})

假设 draft 过采样 A: $p(A) > q(A)$ ，因此 $p(B) < q(B)$ 。残差概率 $\max(q - p, 0) = [0, 1]$ 。

- $P[\text{采到}A] = p(A) \cdot \frac{q(A)}{p(A)} + p(B) \cdot 1 \cdot 0 = q(A)$ (成立)
- $P[\text{采到}B] = p(B) \cdot 1 + p(A) \cdot \left(1 - \frac{q(A)}{p(A)}\right) \cdot 1 = q(B)$ (成立)

分布与直接从 q 采样完全一致。

10.4 实践与扩展

- 配对例子: target 70B / draft 8B; 或 target 8B / draft 1B。
- 让 draft 尽量接近 target (蒸馏)。
- **Medusa**: draft 并行生成多个 token。
- **EAGLE**: draft 使用 target 的高层特征。

10.5 小结

- 靠数学保证从 target 精确采样。
- 利用”检查 vs 生成”的不对称。
- draft model 上有大量创新空间 (涉及训练)。

11. 处理动态负载（系统技巧）

真实流量中 batch 很棘手，原因：

1. 请求在不同时间到达（等凑齐 batch 对早到的请求不公平）。
2. 序列有共享前缀（系统提示、对同一 prompt 采多个样本）。
3. 序列长度不同（padding 浪费）。

11.1 Continuous Batching (Orca)

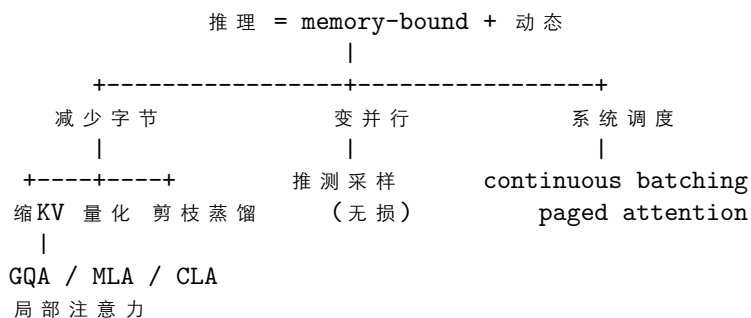
- 问题：训练拿到的是稠密块（batch $\times$ seq）；推理里请求参差不齐，是“锯齿数组”。
- 方案：**iteration-level 调度**——逐步解码，新请求到达就加入 batch，不必等生成完成。
- 问题：batch 要求所有序列同维度，但长度不同。
- 方案：**selective batching**——attention 计算对每条序列分别做；非 attention 计算把所有序列拼接成 $[3 + 9 + 5, H]$ 一起算。

11.2 PagedAttention (vLLM)

- 旧状况：请求来了，按最大长度给 prompt+response 分配一整段 KV cache。
- 问题：碎片化（像硬盘）——实际生成的 token 可能少很多（内部碎片），段之间还有空隙（外部碎片）。
- 方案：借操作系统的分页思想——把一个序列的 KV cache 切成非连续的 **block**。
- 好处：多个序列可以共享 KV cache block（共享系统提示、一个 prompt 采多个响应），前缀共享 + block 级 copy-on-write。
- 其它 vLLM 优化：融合 block 读取与 attention 的 kernel、用最新 kernel (FlashAttention / FlashDecoding)、用 CUDA graph 避免 kernel 启动开销。

12. 全局总结

12.1 一图流



12.2 要点清单

- 推理很重要：实际使用、模型评测、强化学习。
- 与训练特性不同：memory-bound、动态。
- 技巧：新架构、量化、剪枝/蒸馏、推测采样。
- 借鉴系统思想：推测执行、分页。
- 新架构有巨大的改进潜力。

12.3 最容易考/最该记的三件事

1. 算术强度判据: > 295 compute-bound, < 295 memory-bound (H100)。
2. 四个强度公式: Prefill-MLP BS 、Prefill-Attn $S/2$ 、Gen-MLP B 、Gen-Attn < 1 。
3. **batch** 救得了 **MLP**、救不了 **Attention** (因为 attention 强度不含 B)。